CRONOGRAMA DE SPRINTS

Sistema Web para ONG Vira Lata Vira Luxo

Período	03/02/2026 → 07/04/2026 (5 sprints semanais, terça a terça)
Stack	Next.js + Django REST Framework + PostgreSQL (Supabase)
Deploy	Vercel (frontend) + Railway (backend) + Supabase (banco)
Grupo	Kayque Santos - João Pedro Temporim - Igor Rangel - Luis Felipe
Módulos	Autenticação · Cadastro de Animais · Saúde · Financeiro · Relatórios
Requisitos Principais	Autenticação JWT · Testes automatizados · Swagger · Exportação de dados

Visão Geral das Sprints

Sprint	Período	Foco Principal
S1	03/03 (ter) →10/03 (ter)	Modelagem do Banco de Dados, Setup do Ambiente e Wireframes
S2	10/03 (ter) → 17/03 (ter)	Autenticação + Cadastro de Animais + Swagger
S3	17/03 (ter) → 24/03 (ter)	Módulo de Saúde: Vacinas, Vermífugo, Carrapaticida e Alertas
S4	24/03 (ter) → 31/03 (ter)	Módulo Financeiro + Relatórios + Exportação de Dados
S5	31/03 (ter) → 07/04 (ter)	Deploy, Testes Finais, Documentação Completa e Entrega

1. Tecnologias Utilizadas

A stack foi escolhida equilibrando familiaridade do grupo, qualidade técnica e custo zero em produção.

Camada	Tecnologia	Função no projeto
Backend	Python + Django REST Framework	API REST, regras de negócio, autenticação JWT
Frontend	Next.js (React) + Tailwind CSS	Interface web responsiva, consumo da API
Banco de Dados	PostgreSQL via Supabase	Persistência de dados com backups automáticos e painel visual
Autenticação	Django SimpleJWT	Geração e validação de tokens JWT
Documentação	drf-spectacular (Swagger/OpenAPI)	Documentação automática da API em /api/docs/
Testes	pytest-django + pytest-cov	Testes unitários e de integração com relatório de cobertura
Deploy Backend	Railway	Hospedagem gratuita do Django com suporte a PostgreSQL
Deploy Frontend	Vercel	Hospedagem gratuita do Next.js com CI/CD automático via GitHub
Versionamento	Git + GitHub	Controle de versão com branches por feature
Design	Figma	Prototipação de baixa e alta fidelidade das telas

2. Requisitos do Sistema

2.1 Requisitos Funcionais

Requisito Funcional	Descrição
RF01 — Cadastro de Animais	CRUD completo: nome, sexo, porte, nascimento, castração, status e observações
RF02 — Saúde dos Animais	Registro de vacinas (antirrábica, V8/V10, outras), vermífugo e carrapaticida com histórico
RF03 — Alertas de Saúde	Painel exibindo animais com doses vencidas ou vencendo nos próximos 30 dias
RF04 — Controle Financeiro	Lançamento de contas a pagar, registro de pagamentos e listagem por status
RF05 — Dashboard Financeiro	Resumo mensal: total pago, pendente e contas vencidas
RF06 — Exportação de Dados	Exportar lista de animais e contas para CSV via download direto
RF07 — Autenticação	Login/logout com JWT; rotas protegidas; controle de sessão

RF08 — Documentação da API	Swagger UI disponível em /api/docs/ com todos os endpoints descritos

2.2 Requisitos Não-Funcionais

Requisito Não-Funcional	Detalhe	Como atender
RNF01 — Segurança	Autenticação JWT em todas as rotas sensíveis	*Django SimpleJWT + middleware Next.js*
RNF02 — Testabilidade	Cobertura de testes nos módulos principais	*pytest-django + pytest-cov*
RNF03 — Documentação	API auto-documentada e atualizada	*drf-spectacular (OpenAPI 3.0)*
RNF04 — Disponibilidade	Sistema acessível via URL pública	*Railway (backend) + Vercel (frontend)*
RNF05 — Banco de dados	Persistência confiável com backups automáticos	*Supabase (PostgreSQL gerenciado)*
RNF06 — Usabilidade	Interface responsiva e acessível	*Next.js + Tailwind CSS*

3. Cronograma Detalhado por Sprint

SPRINT 1 03/03 (ter) →10/03 (ter)

🎯 **Foco:** Modelagem do Banco de Dados, Setup do Ambiente e Wireframes

Categoria	Tarefa / Entregável	Responsável	Tipo
Banco	Criar diagrama ER com todas as entidades: Animal, Vacina, Vermifugo, Carrapaticida, Conta, Pagamento, Usuario	*Todos*	**Banco**
Banco	Definir relacionamentos, chaves estrangeiras e regras de integridade no diagrama	*Todos*	**Banco**
Banco	Criar projeto Supabase, configurar banco PostgreSQL e testar conexão local	*Dev Backend*	**Backend**
Ambiente	Criar repositório GitHub com branches (main / develop / feature/*) e definir fluxo de PR	*Todos*	**DevOps**
Ambiente	Setup Django + DRF + django-environ + conexão com Supabase (PostgreSQL)	*Dev Backend*	**Backend**
Ambiente	Setup Next.js + Tailwind CSS + ESLint + estrutura de pastas do projeto	*Dev Frontend*	**Frontend**
Ambiente	Configurar variáveis de ambiente (.env) para backend e frontend	*Todos*	**DevOps**
Figma	Wireframes de baixa fidelidade: Login, Dashboard principal, Cadastro de Animal	*Design (Figma)*	**Figma**

Figma	Definir paleta de cores, tipografia e componentes base do design system	*Design (Figma)*	**Figma**

SPRINT 2 10/03 (ter) → 17/03 (ter)

🎯 **Foco:** Autenticação + Cadastro de Animais + Swagger

Categoria	Tarefa / Entregável	Responsável	Tipo
Auth	Implementar login/logout com JWT (SimpleJWT): endpoints /token e /token/refresh	*Dev Backend*	**Backend**
Auth	Criar tela de Login no Next.js com validação e armazenamento seguro do token	*Dev Frontend*	**Frontend**
Auth	Middleware Next.js: proteger rotas privadas, redirecionar usuário não autenticado	*Dev Frontend*	**Frontend**
Animal	Models: Animal (nome, sexo, porte, nascimento, castrado, status, observações)	*Dev Backend*	**Backend**
Animal	API REST: GET /animais, POST, GET/:id, PUT/:id, DELETE/:id (com permissão JWT)	*Dev Backend*	**Backend**
Animal	Tela de listagem de animais com filtros (status, sexo, porte) e busca por nome	*Dev Frontend*	**Frontend**
Animal	Formulário de cadastro e edição de animal com validação client-side	*Dev Frontend*	**Frontend**
Swagger	Configurar drf-spectacular: gerar /api/docs/ com todos os endpoints documentados	*Dev Backend*	**Backend**
Testes	Testes unitários: models Animal + endpoints de autenticação (pytest-django)	*QA / Testes*	**Testes**
Figma	Evoluir protótipo: telas de Login, Listagem e Cadastro de Animal em alta fidelidade	*Design (Figma)*	**Figma**

SPRINT 3 17/03 (ter) → 24/03 (ter)

🎯 **Foco:** Módulo de Saúde: Vacinas, Vermífugo, Carrapaticida e Alertas

Categoria	Tarefa / Entregável	Responsável	Tipo
Saúde	Models: VacinaAntirabica, VacinaV10, OutraVacina, Vermifugo, Carrapaticida (animal + data + próxima)	*Dev Backend*	**Backend**
Saúde	Lógica de recorrência: antirrábica (anual), vermífugo (4/4 meses), carrapaticida (3/3 meses)	*Dev Backend*	**Backend**
Saúde	API REST: CRUD de registros de saúde por animal + endpoint de alertas /saude/alertas/	*Dev Backend*	**Backend**
Saúde	Documentar novos endpoints no Swagger (drf-spectacular)	*Dev Backend*	**Backend**

Saúde	Tela de saúde do animal: histórico de vacinas, vermífugo e carrapaticida por animal	*Dev Frontend*	**Frontend**
Saúde	Painel de alertas: animais com doses vencidas ou vencendo nos próximos 30 dias	*Dev Frontend*	**Frontend**
Testes	Testes unitários: cálculo de próximas datas (casos de borda: data nula, animal novo)	*QA / Testes*	**Testes**
Testes	Testes de integração: endpoint /saude/alertas/ com animais de teste no Supabase	*QA / Testes*	**Testes**
Figma	Prototipar tela de saúde e painel de alertas no Figma	*Design (Figma)*	**Figma**

SPRINT 4 24/03 (ter) → 31/03 (ter)

🎯 **Foco:** Módulo Financeiro + Relatórios + Exportação de Dados

Categoria	Tarefa / Entregável	Responsável	Tipo
Financeiro	Models: Conta (descricao, fornecedor, categoria, valor, vencimento, status), Pagamento	*Dev Backend*	**Backend**
Financeiro	API REST: CRUD de contas, registrar pagamento, listar em aberto/pagas/vencidas	*Dev Backend*	**Backend**
Financeiro	Endpoint de resumo mensal: total pago, total pendente, contas vencidas	*Dev Backend*	**Backend**
Financeiro	Tela de contas: listagem com filtros (status, categoria, período) + formulário de lançamento	*Dev Frontend*	**Frontend**
Financeiro	Dashboard financeiro: cards de resumo + gráfico de contas por mês (Chart.js / Recharts)	*Dev Frontend*	**Frontend**
Relatórios	Exportação de animais para CSV e de contas para CSV via endpoint /export/	*Dev Backend*	**Backend**
Relatórios	Botões de exportação no frontend conectados aos endpoints de download	*Dev Frontend*	**Frontend**
Testes	Testes unitários e de integração: módulo financeiro e endpoints de exportação	*QA / Testes*	**Testes**
Figma	Prototipar módulo financeiro, dashboard e tela de relatórios no Figma	*Design (Figma)*	**Figma**
Swagger	Atualizar documentação Swagger com endpoints financeiros e de exportação	*Dev Backend*	**Backend**

SPRINT 5 1/03 (ter) → 07/04 (ter)

🎯 **Foco:** Deploy, Testes Finais, Documentação Completa e Entrega

Categoria	Tarefa / Entregável	Responsável	Tipo

Deploy	Deploy Django no Railway: configurar variáveis de ambiente e banco Supabase (PostgreSQL)	*Dev Backend*	**DevOps**
Deploy	Deploy Next.js na Vercel: conectar à API Railway, configurar domínio público	*Dev Frontend*	**DevOps**
Deploy	Executar migrations em produção e validar todos os endpoints na URL pública	*Todos*	**DevOps**
Integração	Teste de ponta a ponta: fluxo completo (login → animal → saúde → financeiro → export)	*Todos*	**Integração**
Testes	Corrigir bugs críticos encontrados em produção	*Todos*	**Testes**
Testes	Rodar suite completa de testes e garantir cobertura mínima dos módulos principais	*QA / Testes*	**Testes**
Docs	Finalizar README: instalação local, configuração Supabase, variáveis e deploy	*Todos*	**Docs**
Docs	Revisar diagrama ER, casos de uso e protótipo Figma — versão final	*Todos*	**Docs**
Apresentação	Montar slides: problema, solução, stack, demos das funcionalidades, aprendizados	*Todos*	**Docs**
Entrega	🏁 Revisão final de todos os entregáveis — ENTREGA DO TRABALHO	*Todos*	**Integração**

4. Entregáveis do Projeto

Entregável	Descrição	Sprint
Diagrama ER / DER	Modelo completo do banco com entidades, atributos e relacionamentos	*Sprint 1 → revisão Sprint 5*
Documentação de Requisitos	RF, RNF, casos de uso e regras de negócio (definidos antes das sprints)	*Pré-Sprint → revisão Sprint 5*
Protótipo Figma	Wireframes evoluindo para alta fidelidade módulo a módulo	*Sprints 1→4 (incremental)*
Módulo de Autenticação	Login/logout JWT funcionando no frontend e backend	*Sprint 2*
Módulo de Animais	CRUD completo com listagem, filtros e formulários	*Sprint 2*
Swagger / OpenAPI	Documentação da API disponível em /api/docs/ desde a Sprint 2	*Sprint 2 → atualização contínua*
Módulo de Saúde	Vacinas, vermífugo, carrapaticida com alertas automáticos	*Sprint 3*
Módulo Financeiro	Contas, pagamentos, dashboard e exportação CSV	*Sprint 4*
Suite de Testes	Testes unitários e de integração com relatório de cobertura	*Sprints 2→5*

Sistema em Produção	URL pública funcional: Vercel (frontend) + Railway (backend/API)	*Sprint 5*
README + Documentação Final	Instalação, configuração, arquitetura e decisões técnicas	*Sprint 5*
Slides de Apresentação	Problema, solução, stack, demo das funcionalidades e aprendizados	*Sprint 5*